BRICK BUSTING BALL GAME

PADDLE DESIGN, POWERUPS, SHOW HELP AND HANDLING BALL COLLISIONS WITH BRICK

CODE:

PADDLE:

```
#include "paddle.h"

/** Default constructor for Paddle
 */
Paddle::Paddle()
{

    //initialize animation settings
    width = 200;
    height = 15;
    starting_width = width;
    starting_height = height;
    pen_width = 1;

    //set movement distance for key events
    default_movement_distance = 20;
    starting_movement_distance =
default_movement_distance;

    //set size_change_amount for shrinking and growing
    size_change_amount = 50;

    //set speed_change_amount for faster and slower
    speed_change_amount = 6;
```

```
    //give us starting position
    start_vector.setX(170);
    start_vector.setY(440);
    setPos(mapToParent(start_vector.x(), start_vector.y()));

}

/** Constructor for Paddle. Sets paddle at given x,y position.
 *  @param start_x_position starting x coordinate in scene
 *  @param start_y_position starting y coordinate in scene
 */
Paddle::Paddle(int start_x_position, int start_y_position)
{
    //initialize animation settings
    width = 200;
    height = 15;
    starting_width = width;
    starting_height = height;
    pen_width = 1;

    //set movement distance for key events
    default_movement_distance = 20;
    starting_movement_distance =
default_movement_distance;

    //set size_change_amount for shrinking and growing
    size_change_amount = 50;

    //set speed_change_amount for faster and slower
    speed_change_amount = 6;
```

```
    //give us starting position
    start_vector.setX(start_x_position);
    start_vector.setY(start_y_position);
    setPos(mapToParent(start_vector.x(), start_vector.y()));
}

/** Set bounding rectangle for drawing and collisions
    @returns bounding rectange for shape
 */
QRectF Paddle::boundingRect() const
{
    return QRectF(0,
                  0,
                  width,
                  height);
}

/** Paints object with correct shape in local coordinates.
Colors according to powerup.
    @param *painter QPainter object resposible for painting
the object in the scene
    @param *option provides style options for the item, such
as its state, exposed area and its level-of-detail hints
    @param *widget Optional. If provided, it points to the
widget that is being painted on; otherwise, it is 0.
 */
void Paddle::paint(QPainter *painter, const
QStyleOptionGraphicsItem *option,
          QWidget *widget)
{
```

```
    //color according to paddle speed

    //color faster paddle same color as faster powerup
    if(default_movement_distance >
starting_movement_distance)
        painter->setBrush(Qt::green);

    //color slower paddle same color as slower powerup
    else if(default_movement_distance <
starting_movement_distance)
        painter->setBrush(Qt::red);

    //color normal paddle
    else
        painter->setBrush(Qt::magenta);

    //should match shape in Paddle::shape
    painter->drawRect(0, 0, width, height);

    painter->setBrush(Qt::NoBrush);
}


/** Gives an accurate shape, which is used by
QGraphicsScene to animate.
    @returns path QPainterPath to draw representing shape
 */
QPainterPath Paddle::shape() const
{
    QPainterPath path;
```

```
    //should match shape in Paddle::shape
    path.addRect(0, 0, width, height);
    return path;
}

/** Move the paddle default_movement_distance number of
units to the left
*/
void Paddle::move_left()
{
    //update position based on moving to the left
    QPointF updated_position = mapToParent(-
default_movement_distance,0);

    //if the new position would be out of bounds, don't allow
the movement
    if(!scene()->sceneRect().contains((updated_position)))
    {
        return;
    }
    else
        setPos(updated_position);
}

/** Move the paddle default_movement_distance number of
units to the right
 */
void Paddle::move_right()
{
    //update position based on moving to the right
```

```
    QPointF updated_position =
mapToParent(default_movement_distance,0);

    //adjust for width of paddle
    QPointF updated_adjusted_position =
mapToParent(default_movement_distance+width,0);

    //if the new position would be out of bounds, don't allow
the movement
    if(!scene()-
>sceneRect().contains((updated_adjusted_position)))
    {
        return;
    }
    else
        setPos(updated_position);
}

/** Getter for width member variable
 * @return returns value of width member variable
 */
qreal Paddle::get_width()
{
    return width;
}

/** Getter for height member variable
 * @return returns value of height member variable
 */
qreal Paddle::get_height()
{
```

```
    return height;
}

/** Setter for width member variable
 * @param new_width new width value to set width to
 */
void Paddle::set_width(qreal new_width)
{
    width = new_width;
}

/** Setter for height member variable
 * @param new_height new height value to set height to
 */
void Paddle::set_height(qreal new_height)
{
    height = new_height;
}

/** Decrease width of paddle by size_change_amount
 */
void Paddle::shrink()
{
    width -= size_change_amount;
}

/** Increase width of paddle by size_change_amount
 */
void Paddle::grow()
{
    width += size_change_amount;
```

```
}

/** Decrease default_movement_distance of paddle by
speed_change_amount
 */
void Paddle::slower()
{
   default_movement_distance -= speed_change_amount;
}

/** Increase default_movement_distance of paddle by
speed_change_amount
 */
void Paddle::faster()
{
   default_movement_distance += speed_change_amount;
}

/** Undoes the effect of any powerups by returning paddle
back to starting width, height, and movement_distance
 */
void Paddle::reset_powerups()
{
   width = starting_width;
   height = starting_height;
   default_movement_distance =
starting_movement_distance;
}
```

HANDLING POWER UPS

```
void Level_1::handle_powerups(Brick* hit_brick)
{
   if(hit_brick->is_shrink_powerup())
   {
      paddle_top->shrink();
      paddle_bottom->shrink();
   }
   else if(hit_brick->is_grow_powerup())
   {
      paddle_top->grow();
      paddle_bottom->grow();
   }
   else if(hit_brick->is_slower_powerup())
   {
      paddle_top->slower();
      paddle_bottom->slower();
   }
   else if(hit_brick->is_faster_powerup())
   {
      paddle_top->faster();
      paddle_bottom->faster();
   }
}
```

SHOW HELP

```
/** Show instructions and controls. Pauses game while
displaying, unpauses game after.
 */
void Level_1::show_help()
{
    pause_level();
    QMessageBox::information(this,
                             tr("Instructions and Controls"),
                             tr("Instructions: \n \n - Use your paddles
and the ball to bust all the bricks! \n - Both top and bottom
paddles move together \n - Don't let the ball go past your
paddles! \n - Break powerup bricks to modify your paddles! \n
- Break all the bricks to win! \n - Press the 'Instructions /
Controls' button during gameplay to see all this again! \n -
Close the game window (using the 'x' in the top left of the
window) to exit the game! \n \n Controls: \n \n - Use 'space'
on your keyboard to launch the ball \n - Use 'a' or 'd' on your
keyboard to move around \n - Use 'p' on your keyboard to
pause / unpause \n \n Powerups: \n \n - Blue blocks are normal
\n - Yellow blocks shrink your paddle \n - Cyan blocks grow
your paddle \n - Red blocks slow down your paddle \n - Green
blocks speed up your paddle"));
    unpause_level();
}
```

COLLISION WITH BRICK

```
/** Checks all the currently existing bricks to see if they've
been hit. If they have, it removes them from 'bricks', the
scene, and deletes them.
 */
void Level_1::handle_collisions()
{
   // get an iterator for iterating through the list
   QList<Brick*>::iterator bricks_iterator = bricks.begin();

   // for each brick in bricks
   //NOTE: Not using foreach because am deleting from list
while iterating over it
   while(bricks_iterator!=bricks.end())
   {
      // if something collides with the brick
      if((*bricks_iterator)->is_hit())
      {
         //handle direction of ball bounce
         //duplicate cases bouncing 270 for clarity of division
between cases

         //bouncing off bottom of brick going left / up
         if(((ball->pos().y()) > (*bricks_iterator)->pos().y())
&& (ball->get_movement_vector().x() < 0) && (ball-
>get_movement_vector().y() < 0))
         {
            ball->bounce(270);
         }
```

```
        //bouncing off top of brick going right / down
        else if(((ball->pos().y()+(0.5)*ball->get_radius()) <
(*bricks_iterator)->pos().y()) && (ball-
>get_movement_vector().x() > 0) && (ball-
>get_movement_vector().y() > 0))
        {
           ball->bounce(270);
        }

        //bouncing off left of brick going left / up
        else if((ball->pos().x()+(0.5)*ball->get_radius() <
(*bricks_iterator)->pos().x()) && (ball-
>get_movement_vector().x() > 0) && (ball-
>get_movement_vector().y() < 0))
        {
           ball->bounce(270);
        }



        //otherwise just bounce the ball
        else
        {
           ball->bounce(90);
        }

        //handle powerups from brick
        handle_powerups(*bricks_iterator);

        // scene no longer has ownership, doesn't call delete
        scene->removeItem(*bricks_iterator);
```

```
            // erase from QList bricks, return the iterator at the
new correct position
            // does call delete
            delete (*bricks_iterator);
            bricks_iterator = bricks.erase(bricks_iterator);

        }
        // else move on to next brick
        else
            ++bricks_iterator;
    }
}
```

EXPLANATION

Paddle Class

The Paddle class handles the behavior and functionality of the paddles in the game. There are two paddles, one on the top and one on the bottom, which are controlled by the player.

1. **Default Constructor**:
    - The paddle's initial properties like **width**, **height**, and **movement distance** are set here. It also defines the starting position for the paddle on the screen. The width of the paddle is set to 200 units, and the height is set to 15 units. The initial movement speed (how far the paddle moves per key press) is 20 units.
2. **Constructor with Custom Position**:
    - This constructor allows the paddle's starting position to be set dynamically, enabling flexibility for future changes or different gameplay modes.
3. **Bounding Rectangle**:
    - This method returns a rectangle that bounds the paddle. This rectangle is crucial for detecting collisions between the paddle and other objects, like the ball or bricks.
4. **Painting the Paddle**:
    - The paint method is responsible for rendering the paddle on the screen. It changes the color of the paddle based on its speed. For example, when the paddle is moving faster due to a power-up, it turns green; when it's slower, it turns red; and when it's at a normal speed, it is magenta.
5. **Shape of the Paddle**:

    - This method defines the paddle’s shape for collision detection. In this case, the shape is a simple rectangle that matches the paddle’s dimensions.
6. **Movement Methods**:
    - The paddle can move left or right based on the player's keyboard input. The movement is constrained so that the paddle does not move off-screen.
7. **Size and Speed Modification Methods**:
    - These methods are used to modify the paddle's size and speed in response to various power-ups:
        - **Shrink**: Reduces the paddle’s width.
        - **Grow**: Increases the paddle’s width.
        - **Slower**: Reduces the paddle's movement speed.
        - **Faster**: Increases the paddle's movement speed.
8. **Reset Powerups**:
    - This method resets the paddle back to its original size and speed, undoing the effects of any power-ups.

Handling Power-Ups

The Level_1::handle_powerups method processes the power-ups associated with the bricks. These power-ups are activated when the ball hits certain types of bricks. Each power-up affects the paddles differently:

- **Shrink**: Makes the paddles smaller.
- **Grow**: Makes the paddles larger.
- **Slower**: Slows down the paddle's movement speed.
- **Faster**: Speeds up the paddle's movement speed.

Whenever a brick with a power-up is hit, this method modifies the paddles accordingly.

Show Help Instructions

The show_help method is used to display the instructions and controls of the game to the player. This is done using a message box that provides details about:

- The game's objective.
- How to control the paddles.
- What power-ups do.
- The key controls for starting the game, pausing, and moving the paddles.

When this method is triggered, the game is paused temporarily, allowing the player to read the instructions before resuming the gameplay.

Handling Collisions with Bricks

The handle_collisions method is responsible for detecting when the ball collides with a brick. It iterates through all the bricks in the game and checks whether each brick has been hit by the ball.

1. **Bounce Direction**:
    - When a brick is hit, the ball's trajectory is adjusted based on where it collided with the brick (top, bottom, left, or right). This ensures that the ball bounces off the brick at the appropriate angle.
2. **Power-ups**:
    - If the brick contains a power-up, the corresponding power-up is applied to the paddles (e.g., shrinking, growing, slowing down, or speeding up).
3. **Brick Removal**:

- Once the brick is hit, it is removed from the scene and deleted from memory, ensuring that the game's memory is properly managed.

Summary

The code defines the core mechanics of a **Brick Busting Ball Game**, focusing on the paddle's movement, size and speed changes, handling collisions, and managing power-ups. The key components of the game are:

- **Paddle**: This object handles the movement and rendering of the paddles. It also manages the changes to the paddle's size and speed in response to power-ups.
- **Power-ups**: These are triggered by breaking special bricks and modify the paddles' properties. The power-ups include shrinking, growing, slowing, and speeding up the paddles.
- **Collisions**: The game checks for collisions between the ball and bricks, adjusting the ball's direction and applying any power-ups associated with the brick that was hit.
- **Instructions**: A help system is provided, allowing players to view the controls and game objectives during gameplay.

The game is built using Qt's graphics system to render items in the scene, detect collisions, and animate objects like the paddles and the ball. The overall goal is for players to break all the bricks using the paddles while managing the changes caused by power-ups.